INSTITUTO FEDERAL
Goiás
Câmpus Goiânia

Instituto Federal de Educação, Ciência e Tecnologia de Goiás
Pró-Reitoria de Ensino
Câmpus Goiânia
Especialização em Inteligência Artificial Aplicada

Igor Coutinho Ferreira Nobre

Triagem da gravidade de acidentes em rodovias federais com Multilayer Perceptrons

Classificação multiclasse sobre os dados abertos da PRF (2007 a 2026), com busca de hiperparâmetros via Optuna

Relatório de Disciplina

Professor: Prof. Dr. Sérgio Canuto
Disciplina: Deep Learning – Frente 3.1: Multilayer Perceptrons

Goiânia, 2026.

1 INTRODUÇÃO E OBJETIVO

Quando alguém liga para a Polícia Rodoviária Federal (PRF) para relatar um acidente, a central precisa decidir, com pouca informação, o que enviar: só uma viatura e um guincho, ou também ambulância e resgate. Este trabalho treina uma *Multilayer Perceptron* (MLP) para estimar a gravidade do acidente a partir do que o solicitante consegue informar por telefone: tipo de acidente, quantidade de veículos e pessoas envolvidos, local, horário e condições da via. A previsão serve para ordenar as ocorrências por risco e priorizar o despacho; o modelo apoia a decisão, não a substitui.

A unidade de observação é um acidente (uma linha dos arquivos anuais da PRF). A variável alvo é `classificacao_acidente`, com três classes mutuamente exclusivas: `Sem Vítimas`, `Com Vítimas Feridas` e `Com Vítimas Fatais`. Trata-se, portanto, de um problema de classificação multiclasse. Entre as frentes propostas, foi escolhida a de *Multilayer Perceptrons*.

O código foi escrito em Python 3.12, com PyTorch 2.14 para a rede, scikit-learn 1.9 para o pré-processamento e as métricas, e Optuna 5.0 para a busca de hiperparâmetros. Tudo foi executado em CPU (Apple M1 Pro, 16 GB), com sementes fixas. O notebook, os artefatos e o registro completo da busca estão no repositório `https://github.com/coutinhonobre/Trabalho-Final---Pos-Deep-Learning` (Nobre, 2026).

2 CONJUNTO DE DADOS

2.1 Fonte e variáveis

Foi utilizado o conjunto *Acidentes nas Rodovias Federais (agrupados por ocorrência)*, publicado pela PRF no Portal Brasileiro de Dados Abertos (PRF, 2026): um arquivo CSV por ano, de 2007 a 2026, com uma linha por acidente, somando 2.237.189 registros. As 12 entradas, todas categóricas, estão no Quadro 1, em três grupos. As contagens de pessoas e veículos têm caudas longas (mediana 2, mas até 248 pessoas e 131 veículos num registro) e foram convertidas em faixas, que é também o que um solicitante consegue informar.

2.2 Qualidade dos dados e distribuição do alvo

A base tem poucos problemas: no máximo 70 ausentes por coluna de entrada, 11.697 linhas (0,52%) com alvo ausente ou `Ignorado` e 6 duplicatas exatas; o `id` se repete entre anos e foi ignorado. As contagens de vítimas são coerentes com o rótulo (todo `Com Vítimas Fatais` tem mortos > 0), e os 1.081 acidentes com 10 ou mais veículos são engavetamentos plausíveis, absorvidos pela faixa `4+`. Foram removidas 11.875 linhas (0,53%), restando 2.225.314 acidentes.

O alvo é desbalanceado (Figura 1): `Com Vítimas Feridas` 48,4%, `Sem Vítimas` 46,7% e `Com Vítimas Fatais` 4,9%. A fração de `Sem Vítimas` cai de cerca de 60% até 2014 para cerca de 16% a partir de 2019: uma mudança no procedimento de registro, não no perfil dos acidentes.

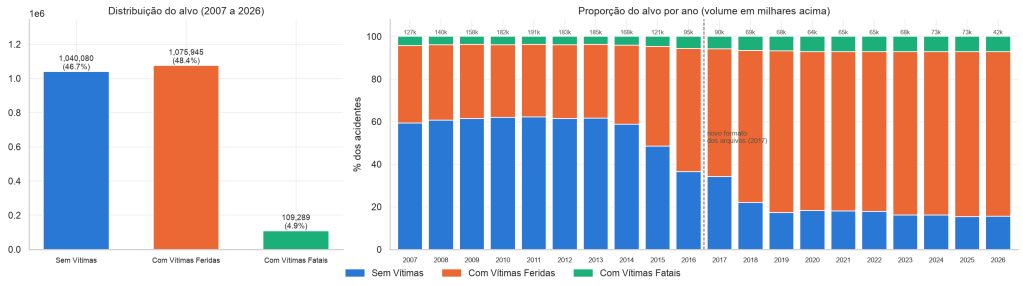
A hora e o dia da semana importam (Figura 2): a taxa de fatais chega a 9% na madrugada, contra cerca de 3% entre 7 h e 17 h, e sobe no sábado (5,8%) e no domingo (6,7%), contra cerca de 4,3% nos demais dias. Isso motivou derivar `periodo_dia` e `fim_de_semana` (esta última agrupando sexta, sábado e domingo, os dias de maior volume). A Figura 3 compara, para cada entrada, a taxa de fatais da categoria mais letal com a da menos letal: as variáveis do relato do solicitante separam muito mais (27,9 p.p. em `tipo_acidente`) que as de contexto da via (4 a 5 p.p.).

Quadro 1 – *Variáveis de entrada*

Grupo	Variável	Categorias	Mais frequente	Observação
Contexto da via	`condicao_metereologica`	10	céu claro (52%)	`ignorado` mantido como valor legítimo
	`tipo_pista`	3	simples (52%)	simples, dupla, múltipla
	`tracado_via`	4	reta (70%)	multirrótulo desde 2017, reduzido à geometria principal
	`fase_dia`	4	pleno dia (57%)	
Quando e onde	`uf`	27	MG (14%)	
	`br`	213 (105 após agrupar raras)	101 e 116 (17% cada)	número da rodovia tratado como rótulo
	`fim_de_semana`	2	não (53%)	derivada de `dia_semana` (sexta a domingo)
	`periodo_dia`	4	tarde (32%)	derivada de `horario` (faixas de 6 h)
Relato do solicitante	`tipo_acidente`	19	colisão traseira (25%)	sinônimos entre formatos unificados
	`macro_causa`	7	desatenção/ manobra (44%)	agrupa as até 75 causas detalhadas
	`veiculos_cat`	4	2 (51%)	faixas 1, 2, 3, 4+
	`pessoas_cat`	5	2 (48%)	faixas 1, 2, 3, 4, 5+

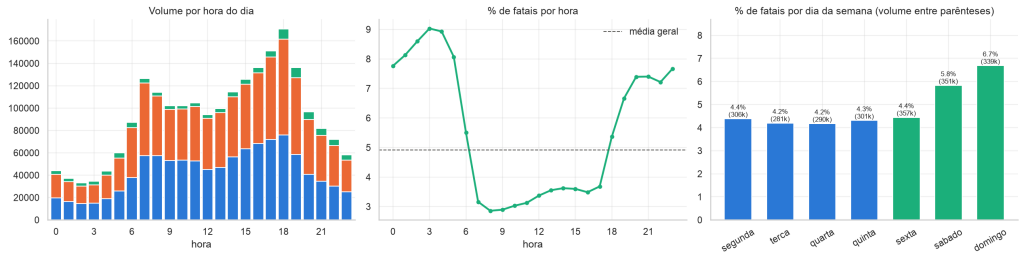
Fonte: Elaborado pelo autor

Figura 1 – *Distribuição do alvo e proporção por ano*



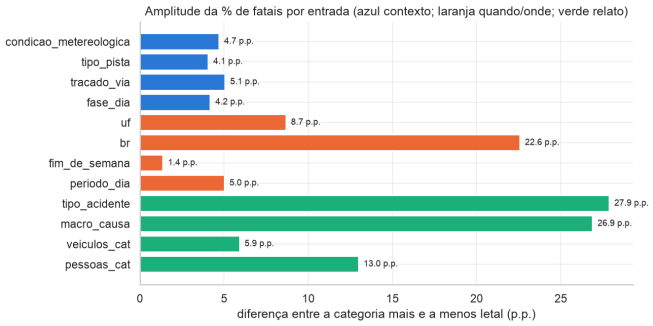
Fonte: Elaborado pelo autor.

Figura 2 – *Volume por hora e taxa de fatais por hora e por dia da semana*



Fonte: Elaborado pelo autor.

Figura 3 – *Amplitude da taxa de fatais por entrada*



Fonte: Elaborado pelo autor.

2.3 Seleção de atributos e risco de vazamento

As contagens de vítimas (`mortos`, `feridos`, `ilesos` etc.) definem o rótulo e foram excluídas: seriam vazamento direto. Para que o modelo não aprendesse o período de registro em vez da gravidade, saíram também o `ano` e os campos que só existem a partir de 2017 (coordenadas, `regional`, `delegacia`, `uop`), e a causa detalhada, cujo vocabulário cresce de 11 valores em 2016 para 75 em 2023, foi agrupada em 7 macrocausas. `km`, `municipio` e `sentido_via` saíram por cardinalidade alta ou pouca informação depois de `uf` e `br`.

2.4 Divisão dos dados e pré-processamento

Os dados foram divididos em 70% treino (1,56 milhão), 15% validação (334 mil) e 15% teste (334 mil), com estratificação pelo alvo. A divisão não é cronológica porque cada acidente é um evento independente e não há unidades com várias observações; a validação cruzada não foi usada pelo custo com 2,2 milhões de linhas. O teste é usado uma única vez.

As entradas foram codificadas com `OneHotEncoder` ajustado somente no treino; categorias com menos de 200 ocorrências ou nunca vistas vão para uma coluna "infrequente". O resultado são 194 colunas binárias, sem necessidade de normalização. Para o desbalanceamento, a perda recebe pesos de classe calculados no treino, com raiz quadrada aplicada (0,845 / 0,830 / 2,605): o peso integral (≈6,8 para fatais) faria o modelo prever fatal em excesso.

3 ARQUITETURA, ESPAÇO DE BUSCA E PROTOCOLO DE HIPERPARAMETRIZAÇÃO

3.1 Arquitetura e treinamento

A MLP foi implementada em PyTorch como uma sequência configurável de blocos `Linear` → `ReLU` → `Dropout` (Código 1): 194 neurônios de entrada (as colunas *one-hot*), camadas ocultas definidas pela busca e 3 neurônios de saída, um por classe, com softmax embutida na `CrossEntropyLoss` ponderada, a configuração usual para classificação multiclasse (Géron, 2019). A ReLU não satura para valores positivos e é barata (Goodfellow; Bengio; Courville, 2016); o otimizador é o Adam; a inicialização é a padrão do PyTorch, já que os esquemas de Glorot e He tratam um problema de redes profundas que não aparece com 1 a 3 camadas. A Figura 4 mostra o modelo final.

Uma única função de treino, por épocas e *mini-batches* com retropropagação do gradiente (Nielsen, 2015), é usada na busca e nos modelos finais, com quatro regularizações: dropout após cada ReLU, L2 (`weight_decay`), L1 (somada à perda como $\lambda \sum |w|$) e parada antecipada pelo Macro F1 de validação, que restaura os pesos da melhor época (paciência 4 na busca, 5 no treino final). A normalização por lote (BatchNorm) foi avaliada (Seção 4.3), mas não entrou nos modelos finais.

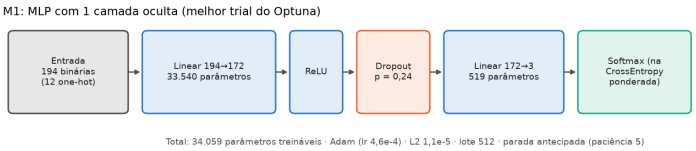
Código 1 Definição da MLP

```python
class MLP(nn.Module):
    def __init__(self, n_in, n_out, hidden_units, dropout=0.0, batch_norm=False):
        super().__init__()
        layers, prev = [], n_in
        for h in hidden_units:                          # camadas ocultas
            layers += [nn.Linear(prev, h)] + ([nn.BatchNorm1d(h)] if batch_norm else []) \
                      + [nn.ReLU(), nn.Dropout(dropout)]
            prev = h
        layers.append(nn.Linear(prev, n_out))           # saída: 1 logit por classe
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)
```

Fonte: Elaborado pelo autor.

Figura 4 – *Arquitetura do modelo final (M1)*



Fonte: Elaborado pelo autor.

3.2 Hiperparametrização: espaço de busca e configuração do estudo

A estratégia foi a busca bayesiana (*Bayesian Search*), com o Optuna. Ao contrário do *grid search*, que percorre toda a grade, e do *random search*, que sorteia às cegas, ela usa os resultados anteriores para escolher a próxima tentativa: o amostrador TPE (*Tree-structured Parzen Estimator*) concentra as tentativas nas regiões do espaço que vêm dando bons resultados. Isso importa aqui porque cada tentativa treina uma rede e o espaço tem sete hiperparâmetros.

Foram 25 tentativas, cada uma treinada numa subamostra estratificada de 300 mil linhas por até 15 épocas e avaliada na validação completa; o `MedianPruner` interrompe, a partir da 5ª época, as tentativas abaixo da mediana. A métrica otimizada foi o Macro F1 de validação, que dá o mesmo peso às três classes. O Quadro 2 apresenta o espaço de busca e a justificativa de cada intervalo; otimizador, ativação e inicialização ficaram fora da busca por baixa influência esperada numa MLP rasa com entradas binárias.

3.3 Da hiperparametrização ao modelo final e protocolo de avaliação

A Figura 5 resume o caminho até o modelo final. As 25 redes da busca só ranqueiam as combinações; as 3 melhores com arquiteturas distintas (M1, M2 e M3) são treinadas do zero com toda a base de treino (até 30 épocas, paciência 5) e avaliadas uma única vez no teste. A subamostra de 300 mil linhas deixou a busca em 6 minutos, e o Macro F1 de validação passou de 0,600 na busca para 0,605 com a base inteira, o que indica que o ranqueamento se manteve.

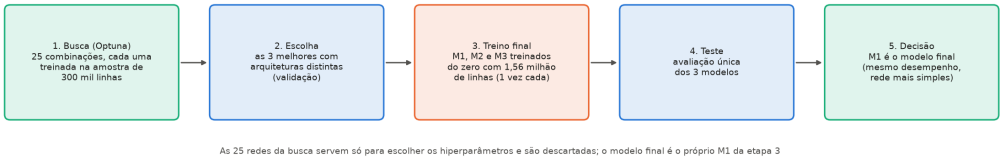
A métrica principal é o Macro F1; as complementares são acurácia, precisão e *recall* macro e matriz de confusão. O custo é medido por parâmetros, épocas e tempos de treino e inferência. Com as mesmas partições e o mesmo *encoder*, foram feitos ainda um experimento controlado de largura × profundidade (um *grid search* de 3 profundidades × 3 larguras, com o resto fixo), a comparação técnica por técnica das regularizações, *baselines* (incluindo regressão logística com o `C` escolhido numa grade de quatro valores) e a ablação por grupo de entradas.

Quadro 2 – *Espaço de busca e justificativa dos intervalos*

Hiperparâmetro	Intervalo	Justificativa
Taxa de aprendizado	10^{-4} a $5 \cdot 10^{-3}$, escala log	abaixo do mínimo a rede não converge no orçamento de épocas; acima do máximo o Adam oscila
Camadas ocultas	1 a 3	uma camada basta pelo teorema da aproximação universal (Hornik; Stinchcombe; White, 1989); uma segunda pode representar a mesma função com menos neurônios (Stathakis, 2009)
Neurônios por camada	16 a 256, escala log, independente por camada	de uma compressão forte até cerca de 1,3 vez as 194 entradas; permite pirâmide ou mesma largura
Tamanho do lote	{256, 512, 1024, 2048}	lotes maiores aceleram cada época; menores introduzem ruído que regulariza
Dropout	0 a 0,5	faixa usual de 10% a 50%, com a opção de desligar
L2	10^{-6} a 10^{-2}, escala log	no M1 treinado $\sum w^2 \approx 447$; com $\lambda = 10^{-2}$ o termo já vale ≈2,2, três vezes a entropia cruzada
L1	desligado ou 10^{-7} a 10^{-4}, escala log	no M1 treinado $\sum \lvert w \rvert \approx 2.900$; com $\lambda = 10^{-3}$ o termo já valeria quatro vezes a entropia cruzada

Fonte: Elaborado pelo autor

Figura 5 – *Da busca de hiperparâmetros ao modelo final*



Fonte: Elaborado pelo autor.

4 RESULTADOS DE VALIDAÇÃO E SELEÇÃO DOS MODELOS

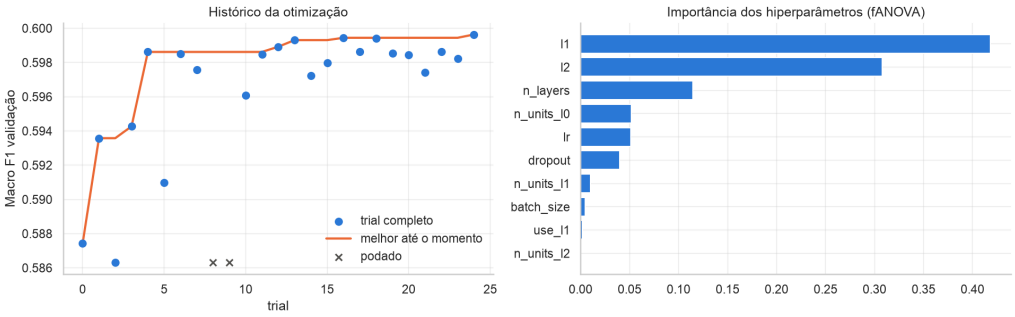
4.1 Resultados da hiperparametrização

As 25 tentativas levaram 6,2 minutos (23 completas, 2 podadas). A Figura 6 mostra a evolução da busca e a importância dos hiperparâmetros; a Tabela 1, as cinco melhores tentativas. O espaço é pouco sensível perto do ótimo (as 17 melhores ficam entre 0,597 e 0,600), e o que mais prejudica é a regularização forte: as piores tentativas combinam L2 alto ($> 10^{-3}$) com taxa de aprendizado baixa, e L1 e L2 são os hiperparâmetros mais importantes na fANOVA. O lote de 512 dominou a partir da tentativa 6 e o sobreajuste é pequeno em todas (*gap* de Macro F1 abaixo de 0,016).

4.2 Efeito da largura e da profundidade

Para isolar o efeito da arquitetura, 9 redes foram treinadas variando só a profundidade (1, 2 ou 3 camadas) e a largura (32, 128 ou 256 neurônios), com os demais hiperparâmetros fixos (Figura 7). A largura importa mais que a profundidade: com 32 neurônios as redes ficam entre 0,595 e 0,597 (subajuste leve), de 32 para 128 o ganho é de até 0,004, e de 128 para 256, de no máximo 0,001. Mais camadas não ajudam: com 128 ou 256 neurônios,

Figura 6 – *Histórico da busca e importância dos hiperparâmetros (fANOVA)*



Fonte: Elaborado pelo autor.

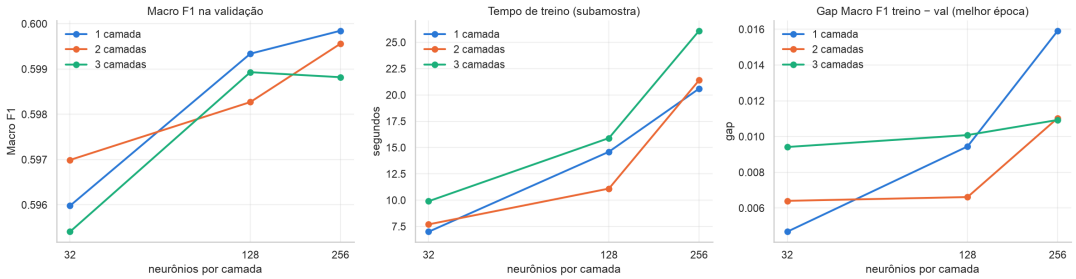
Tabela 1 – *Cinco melhores tentativas do Optuna (Macro F1 na validação)*

Tentativa	Macro F1 val	Camadas ocultas	Dropout	Taxa de aprendizado	L2	L1	Parâmetros	Gap F1
24	**0,5996**	172	0,24	$4{,}6 \cdot 10^{-4}$	$1{,}1 \cdot 10^{-5}$	—	34.059	0,008
16	0,5994	215 → 25	0,32	$7{,}2 \cdot 10^{-4}$	$1{,}2 \cdot 10^{-5}$	$1{,}3 \cdot 10^{-7}$	47.403	0,016
18	0,5994	219 → 17	0,40	$9{,}3 \cdot 10^{-4}$	$8{,}0 \cdot 10^{-6}$	$1{,}1 \cdot 10^{-7}$	46.499	0,008
13	0,5993	228 → 19	0,17	$4{,}8 \cdot 10^{-4}$	$6{,}0 \cdot 10^{-5}$	—	48.871	0,014
12	0,5989	253 → 19	0,14	$3{,}0 \cdot 10^{-4}$	$5{,}0 \cdot 10^{-5}$	—	54.221	0,012

Fonte: Elaborado pelo autor. Lote de 512 em todas. Gap F1 = Macro F1 de treino menos o de validação, na melhor época.

1 camada é igual ou melhor que 2 e 3, que atingem o melhor resultado na época 4 ou 5 e depois só aumentam o *gap*. A rede 3 × 256 tem 182 mil parâmetros e não supera a 1 × 256, com 51 mil. A diferença entre a melhor e a pior das 9 redes é de 0,0044 de Macro F1.

Figura 7 – *Experimento controlado de largura × profundidade (Macro F1 de validação, tempo e gap)*



Fonte: Elaborado pelo autor.

4.3 Efeito da regularização

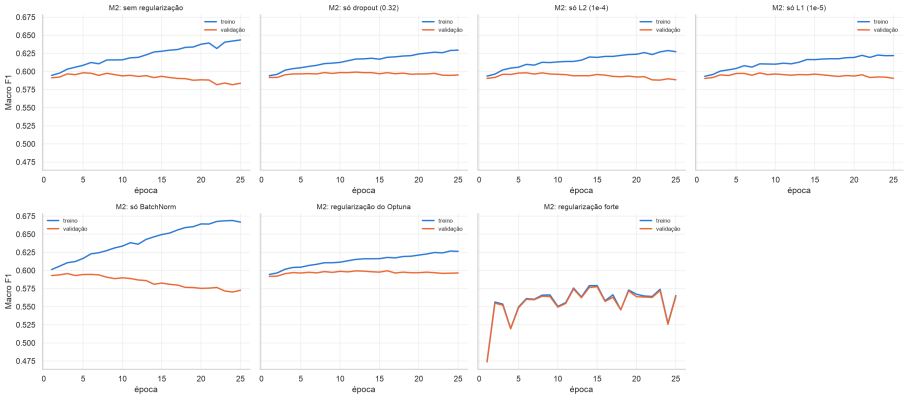
A arquitetura de M2 foi treinada por 25 épocas sem parada antecipada, com cada técnica isolada e com três referências (Tabela 2 e Figura 8). Sem regularização a rede memoriza o treino (*gap* de 0,060 e validação caindo após a 5ª época). O dropout é o que mais ajuda, e a combinação do Optuna é essencialmente esse dropout. L2 e L1 moderados reduzem o *gap* sem melhorar a validação. O BatchNorm piora o sobreajuste (*gap* de 0,094) e por isso ficou fora dos finais. A regularização forte elimina o *gap* à custa do desempenho: subajuste. Simulando a parada antecipada sobre essas curvas, a paciência 1 para cedo demais e perde até 0,008; com paciência 4 ou 5 as três configurações recuperam o melhor valor das 25 épocas, o que justifica os valores usados.

Tabela 2 – *Regularização técnica por técnica (25 épocas, sem parada antecipada)*

Configuração	Macro F1 treino (última)	Macro F1 val (última)	Melhor Macro F1 val (época)	Gap (última)	Diagnóstico
Sem regularização	0,644	0,584	0,598 (5)	**0,060**	sobreajuste: o treino sobe sem parar e a validação cai depois da 5ª época
Só dropout (0,32)	0,630	0,595	0,599 (12)	0,034	técnica isolada mais eficaz
Só L2 (10^{-4})	0,627	0,589	0,598 (6)	0,039	reduz o *gap*, mas a validação ainda cai
Só L1 (10^{-5})	0,622	0,591	0,598 (8)	0,031	efeito parecido com o do L2
Só BatchNorm	0,667	0,573	0,596 (3)	**0,094**	piora o sobreajuste
Combinação do Optuna (dropout 0,32, L2 $1{,}2 \cdot 10^{-5}$, L1 $1{,}3 \cdot 10^{-7}$)	0,626	0,597	**0,600** (16)	0,030	equilíbrio: menor *gap* sem subajuste e melhor validação
Regularização forte (dropout 0,5, L2 10^{-2})	0,565	0,564	0,578 (15)	0,001	subajuste: treino e validação colados e baixos

Fonte: Elaborado pelo autor.

Figura 8 – *Macro F1 de treino e validação por época, por técnica de regularização*



Fonte: Elaborado pelo autor.

4.4 Seleção dos modelos finalistas

As três melhores tentativas com arquiteturas distintas (24, 16 e 18 na Tabela 1) definem os finalistas M1, M2 e M3 (Quadro 3). A busca convergiu para redes rasas e largas: uma camada de 172 neurônios ou duas camadas em formato pirâmide (cerca de 215 e 20 neurônios), sempre com dropout entre 0,24 e 0,40, L2 em torno de 10^{-5}, L1 desligado ou quase nulo e lote 512. Três variações são comparadas para verificar se a segunda camada traz algum ganho no treino com a base completa.

Quadro 3 – *Arquiteturas finalistas resultantes da hiperparametrização*

Modelo	Tentativa	Arquitetura	Parâmetros	Dropout	Taxa de aprendizado	L2	L1
M1	24	$194 \to 172 \to 3$	34.059	0,24	$4{,}6 \cdot 10^{-4}$	$1{,}1 \cdot 10^{-5}$	—
M2	16	$194 \to 215 \to 25 \to 3$	47.403	0,32	$7{,}2 \cdot 10^{-4}$	$1{,}2 \cdot 10^{-5}$	$1{,}3 \cdot 10^{-7}$
M3	18	$194 \to 219 \to 17 \to 3$	46.499	0,40	$9{,}3 \cdot 10^{-4}$	$8{,}0 \cdot 10^{-6}$	$1{,}1 \cdot 10^{-7}$

Fonte: Elaborado pelo autor. Arquitetura: entrada → camadas ocultas → saída; ReLU nas ocultas, softmax na saída, Adam e lote 512 em todos

5 AVALIAÇÃO NO CONJUNTO DE TESTE E CUSTO VERSUS EFETIVIDADE

5.1 Desempenho e custo dos modelos finais

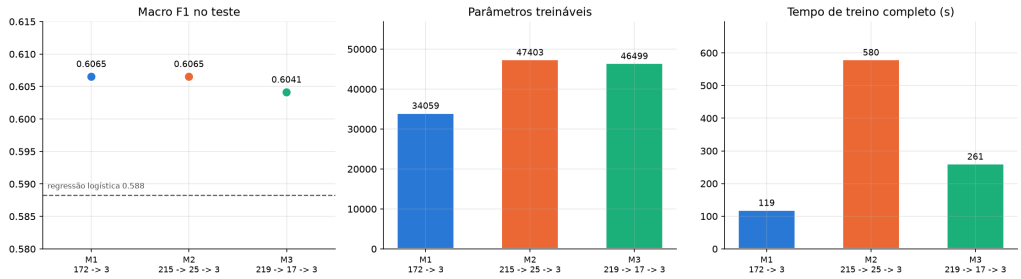
Treinados com toda a base de treino e avaliados uma única vez no teste, M1 e M2 empatam em Macro F1 (0,6065) e M3 fica 0,002 abaixo (Tabela 3 e Figura 9). As diferenças são menores que a variação esperada entre execuções, e o fator decisivo é o custo: M1 foi escolhido por ter o mesmo desempenho com a menor rede (28% menos parâmetros que M2, uma única camada oculta) e um quinto do tempo de treino de M2. A inferência é desprezível nos três: cerca de 0,1 s para 334 mil acidentes em CPU.

Tabela 3 – *Desempenho e custo dos modelos finais no teste*

Modelo	Parâmetros	Épocas (melhor)	Treino (s)	Inferência (ms)	Acurácia	Precisão macro	*Recall* macro	Macro F1	Macro F1 val
M1	34.059	22 (17)	119	126	**0,7029**	0,6020	0,6140	**0,6065**	0,6046
M2	47.403	13 (8)	580	98	**0,7029**	0,6034	0,6142	**0,6065**	0,6041
M3	46.499	8 (3)	261	98	0,7023	0,6051	0,6077	0,6041	0,6017

Fonte: Elaborado pelo autor. Inferência sobre os 333.798 exemplos do teste, em CPU.

Figura 9 – *Macro F1 no teste, parâmetros e tempo de treino dos modelos finais*



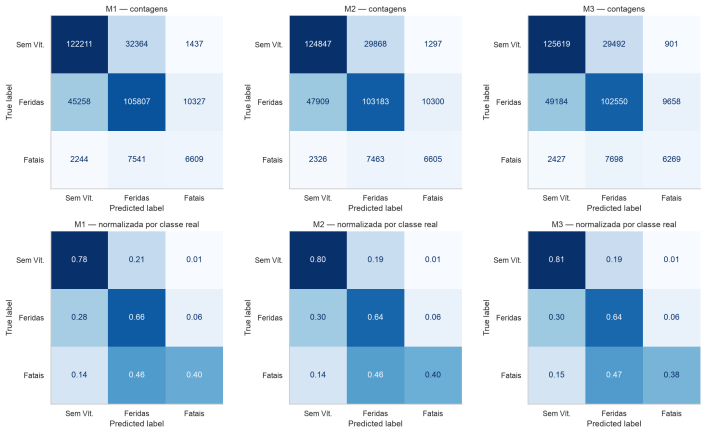
Fonte: Elaborado pelo autor.

A Figura 10 mostra as matrizes de confusão do M1. `Sem Vítimas` é a classe mais fácil (*recall* de 78%); o erro mais comum é entre `Sem Vítimas` e `Com Vítimas Feridas`, que têm relatos parecidos. Dos acidentes fatais, 40% são reconhecidos (precisão 0,36, F1 0,38), 46% vão para `Com Vítimas Feridas` e 14% para `Sem Vítimas`: 86% dos fatais são sinalizados como "com vítima", o que já aciona o socorro, e os 14% restantes são o erro mais grave do modelo.

5.2 Curvas de treino e validação

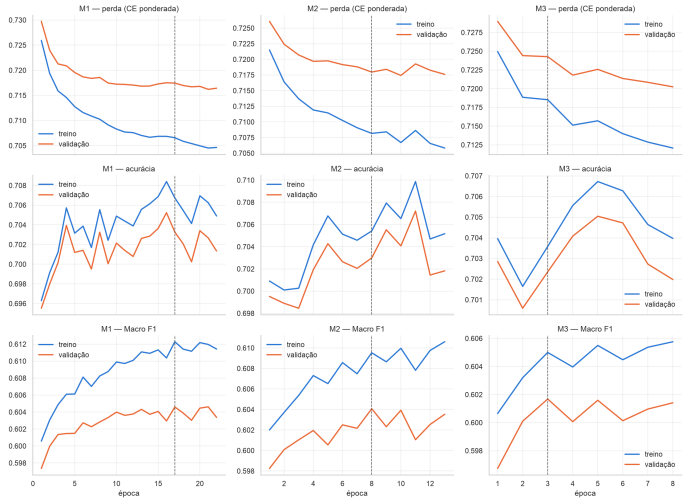
Nas curvas dos três finalistas (Figura 11), a perda de treino e a de validação caem juntas, e o Macro F1 de validação para de melhorar e oscila (melhor época 17 no M1, 8 no M2 e 3 no M3), o que dispara a parada antecipada. O sobreajuste é leve: o *gap* de Macro F1 na melhor época fica entre 0,003 e 0,008, menor no M3, que tem o maior dropout. Não há subajuste: o teto de cerca de 70% de acurácia vem das entradas (Seção 5.3), não da capacidade da rede.

Figura 10 – *Matrizes de confusão do M1 no teste, em contagens e normalizadas pela classe real*



Fonte: Elaborado pelo autor.

Figura 11 – *Perda, acurácia e Macro F1 por época dos finalistas; a linha tracejada marca a melhor época*



Fonte: Elaborado pelo autor.

5.3 Comparação com *baselines* e ablação por grupo de entradas

A MLP supera a regressão logística (com `C` escolhido na validação) em 1,8 p.p. de acurácia e de Macro F1 (Tabela 4): um ganho real, mas pequeno, vindo da combinação não linear das mesmas entradas. A ablação (Figura 12) mostra de onde vem o desempenho: só com o contexto da via a MLP empata com a tabela de frequências (52,9%) e nunca prevê fatal; local e horário acrescentam 2,9 p.p.; o relato do solicitante acrescenta 14,6 p.p. e é o que faz a classe fatal ser reconhecida (*recall* de 0 para 0,40). As entradas pesam muito mais que a arquitetura.

6 LIMITAÇÕES E CONCLUSÕES

6.1 Limitações

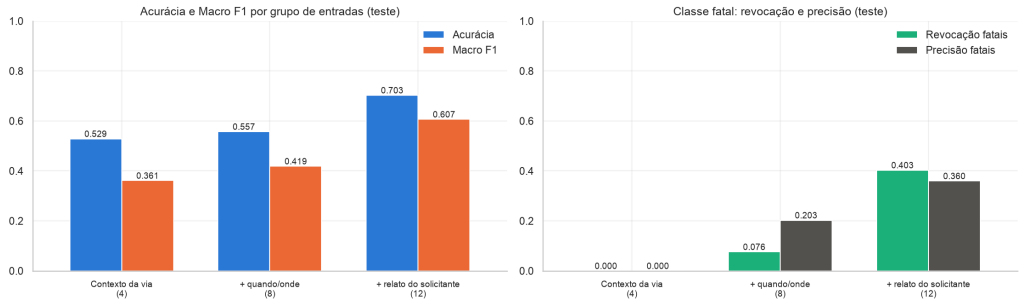
- **Dois regimes de registro.** A categoria `nao informada` de `macro_causa` ainda marca o período anterior a 2017; um teste com os anos mais recentes mediria melhor o uso futuro.

Tabela 4 – *Comparação com baselines no teste*

Modelo	Acurácia	Macro F1
Classe majoritária	0,484	0,217
Sorteio proporcional às classes	0,455	0,334
Tabela de frequências só com o contexto da via (414 combinações)	0,529	0,361
Regressão logística (12 entradas, $C = 1$)	0,685	0,588
MLP M1	**0,703**	**0,607**

Fonte: Elaborado pelo autor.

Figura 12 – *Arquitetura de M1 treinada com grupos crescentes de entradas (teste)*



Fonte: Elaborado pelo autor.

- **Classe fatal.** Com 40% de *recall*, o modelo deixa passar 6 em cada 10 fatais: serve para priorizar, nunca para negar socorro.
- **Qualidade do relato.** O modelo supõe que o solicitante informa corretamente tipo de acidente, pessoas e veículos, justamente as entradas mais importantes.
- **Uma única divisão.** Sem validação cruzada não há intervalo de confiança; diferenças de 0,002 entre os finalistas não são significativas.
- **Escolhas não otimizadas.** O expoente 0,5 dos pesos de classe foi fixado, e a busca usou uma subamostra de 300 mil linhas.

6.2 Conclusões

Uma MLP com uma camada oculta de 172 neurônios (34 mil parâmetros) classifica a gravidade de acidentes em rodovias federais com 70,3% de acurácia e Macro F1 de 0,607 no teste, contra 48% da classe majoritária e 68,5% da regressão logística.

A hiperparametrização mostrou que o desempenho é pouco sensível à arquitetura perto do ótimo e que a regularização forte é o que mais prejudica. A largura importa até cerca de 128 neurônios e mais camadas não ajudam neste problema tabular. O dropout foi a regularização mais eficaz, o BatchNorm piorou o sobreajuste, e a parada antecipada com paciência de 4 ou 5 épocas protegeu o resultado em todos os casos.

O principal limite está nas entradas, não na rede: o relato do solicitante responde por cerca de 15 pontos percentuais de acurácia. Como trabalhos futuros ficam tratar as classes como ordinais, definir o limiar de alerta pelo custo de cada erro e avaliar o modelo num teste temporal.

REFERÊNCIAS

GÉRON, Aurélien. **Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: concepts, tools, and techniques to build intelligent systems**. 2. ed. Sebastopol: O'Reilly, 2019.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.

HORNIK, Kurt; STINCHCOMBE, Maxwell; WHITE, Halbert. Multilayer feedforward networks are universal approximators. **Neural Networks**, v. 2, n. 5, p. 359–366, 1989.

NIELSEN, Michael A. **Neural networks and deep learning**. Determination Press, 2015. Disponível em: `http://neuralnetworksanddeeplearning.com/`. Acesso em: 14 out. 2026.

NOBRE, Igor Coutinho Ferreira. **Trabalho-Final---Pos-Deep-Learning: triagem da gravidade de acidentes em rodovias federais com Multilayer Perceptrons**. 2026. Disponível em: `https://github.com/coutinhonobre/Trabalho-Final---Pos-Deep-Learning`. Acesso em: 14 out. 2026.

POLÍCIA RODOVIÁRIA FEDERAL. **Acidentes nas rodovias federais (agrupados por ocorrência)**. 2026. Portal Brasileiro de Dados Abertos. Disponível em: `https://dados.gov.br/dados/conjuntos-dados/sinistros-de-transito-agrupados-por-ocorrencia`. Acesso em: 14 out. 2026.

STATHAKIS, Demetris. How many hidden layers and nodes? **International Journal of Remote Sensing**, v. 30, n. 8, p. 2133–2147, 2009.